

say it shorter

digitalisierung und programmierung · prof. dr. nicolas meseth

- 1 why anything shrinks
- 2 two tricks, and a zip
- 3 the picture

part 1

why anything shrinks

nothing is gone.
only the air between.



image: ai-generated (gpt-image-2)

why anything compresses at all

die sonne scheint und die sonne waermt und die sonne geht unter

the same nine characters, three times

and german text uses e and n constantly, q and x almost never

every regularity is room the usual notation gives away

the information stays. the description gets shorter.

you have seen this before

symbols and information: morse

e	·	the most frequent letter
t	—	
q	— — · —	one of the rarest
j	· — — —	

code systems: three ways out

fixed length

easy to read: every letter is three bits

and the e pays as much as the q

what we left open: a rule for the short codes

the frequent one gets the short code. now with a recipe.

part 2

two tricks, and a zip

trick one: point back

die sonne scheint und die sonne waermt und die sonne geht unter

63 characters

die sonne scheint und (22 back, 9 long) waermt und (21 back, 9 long) geht unter

45 characters and 2 pointers

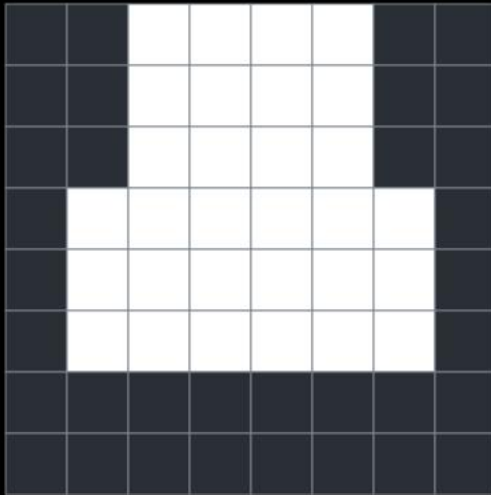
a pointer costs about as much as two characters.

the packer keeps the last few thousand characters in mind and looks for repeats.

say it once. then say where it was.

the same trick on a picture: run length

8 x 8 points, one bit each: 8 bytes



row by row, count equal points:

2 dark, 4 light, 2 dark

1 dark, 6 light, 1 dark

8 dark

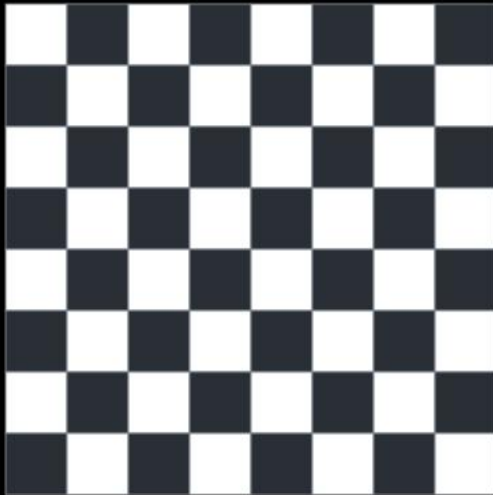
one byte per run: 1 bit colour, 7 bits length

6 bytes instead of 8

the bigger the areas, the more it saves.

run length: the bad case

the same 8 bytes



every point differs from its neighbour:

64 runs of length one
one byte each

a byte that described eight points now describes one

64 bytes instead of 8

every method makes some files bigger. yours too.

trick two: count first

a b r a c a d a b r a

11 letters, 5 different

a × 5 b × 2 r × 2 c × 1 d × 1

fixed length, as in [code systems](#):

5 letters -> 3 bits each

11 * 3 = 33 bit

the same code for every letter: the a pays as much as the d.

build the tree

a 5

b 2 r 2 c 1 d 1

join the two rarest, write the sum:

$$c \ 1 + d \ 1 = 2$$

$$b \ 2 + r \ 2 = 4$$

$$2 + 4 = 6$$

$$a \ 5 + 6 = 11$$

always join the two rarest. that is the whole algorithm.

build the tree



join the two rarest, write the sum:

$$c \ 1 + d \ 1 = 2$$

$$b \ 2 + r \ 2 = 4$$

$$2 + 4 = 6$$

$$a \ 5 + 6 = 11$$

always join the two rarest. that is the whole algorithm.

build the tree



join the two rarest, write the sum:

$$c \ 1 + d \ 1 = 2$$

$$b \ 2 + r \ 2 = 4$$

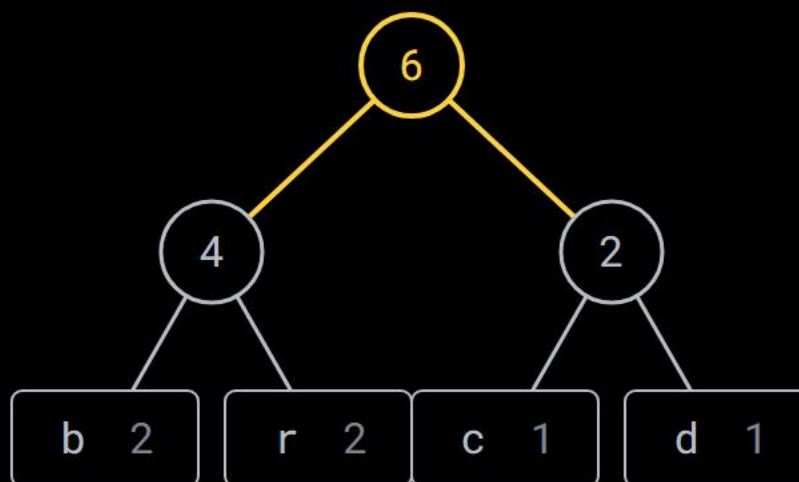
$$2 + 4 = 6$$

$$a \ 5 + 6 = 11$$

always join the two rarest. that is the whole algorithm.

build the tree

a 5



join the two rarest, write the sum:

$$c\ 1 + d\ 1 = 2$$

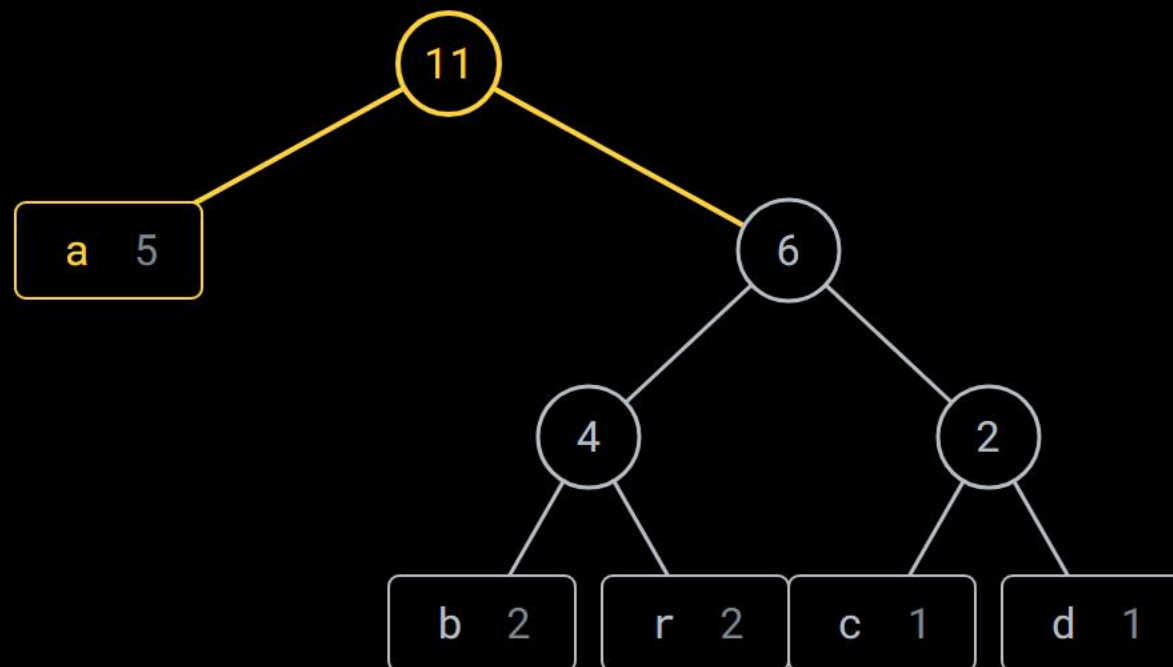
$$b\ 2 + r\ 2 = 4$$

$$2 + 4 = 6$$

$$a\ 5 + 6 = 11$$

always join the two rarest. that is the whole algorithm.

build the tree



join the two rarest, write the sum:

$$c\ 1 + d\ 1 = 2$$

$$b\ 2 + r\ 2 = 4$$

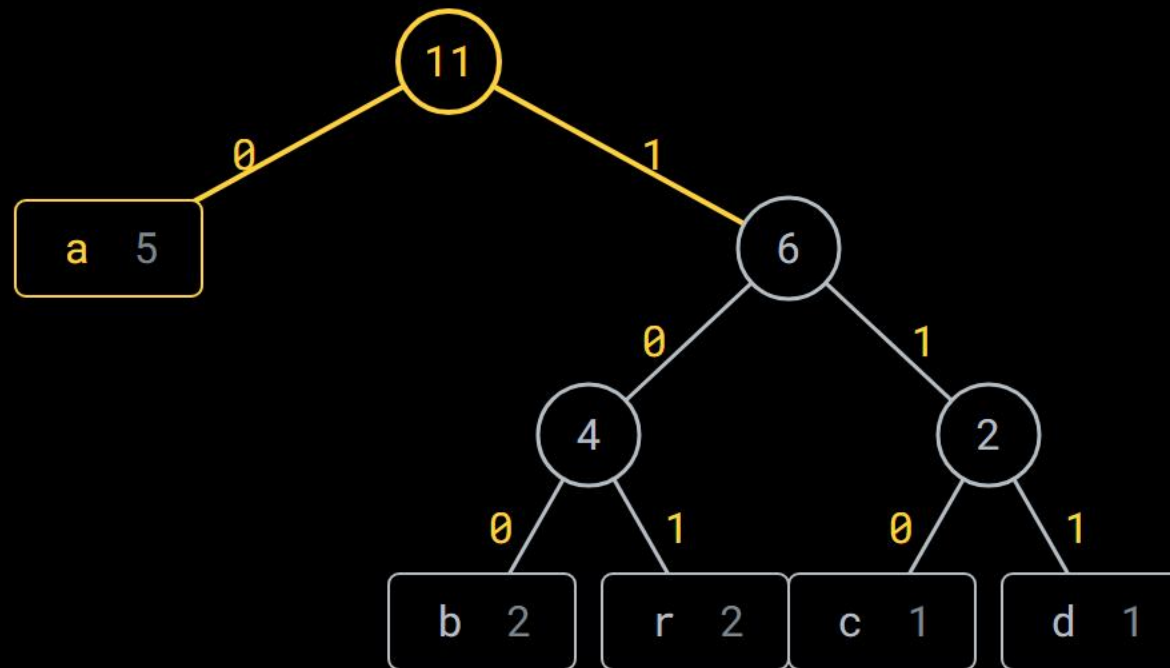
$$2 + 4 = 6$$

$$a\ 5 + 6 = 11$$

11 letters: the root counts the whole word

always join the two rarest. that is the whole algorithm.

read the codes off the tree



left 0, right 1:

a 0
b 100
r 101
c 110
d 111

0100 1010 1100 1110 1001 010

$5*1 + 6*3 = 23 \text{ bit}$

23 bits instead of 33. nothing lost.

why it reads itself

0100101

a b r

read from the left. as soon as a code is complete, the next one starts.

letters sit on leaves only

so no code is the beginning of another

0 is a leaf: nothing else starts with 0

the prefix trap: $a = 0$, $b = 01$

01 reads as b, or as a followed by something. a is not a leaf there.

no code is the start of another. the prefix trap from code systems, avoided by construction.

and that is a zip

zip

1 point back

lempel and ziv, 1977

2 count first

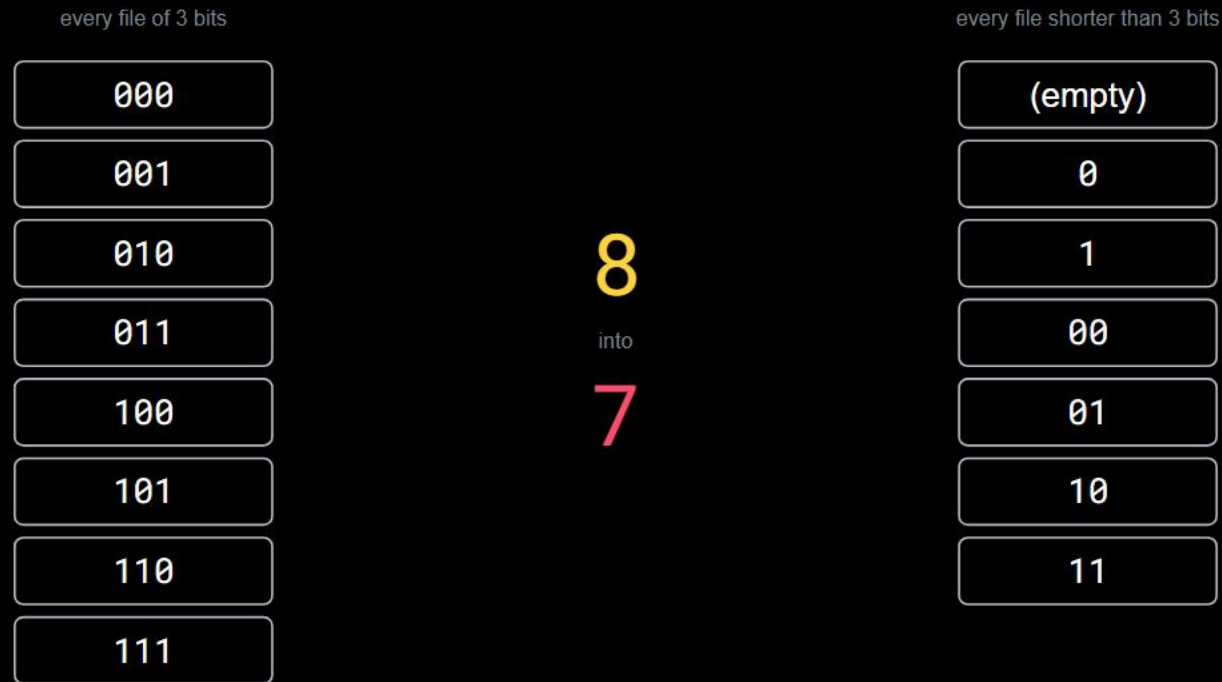
huffman, 1952

the pair is called deflate. png is the same pair with a picture in front.



every zip you ever opened did these two tricks. nothing else.

and that is not bad luck



two of them have to share one shorter file, and then unpacking cannot be unambiguous

no method shrinks everything. counting says so.

part 3

the picture

the parrot, honestly

raw, $256 \times 256 \times 3$



196 608 bytes

44 hours at 10 bit/s

png, lossless



93 107 bytes

21 hours at 10 bit/s

your payload



2 048 bytes

27 minutes at 10 bit/s

the sliver at the bottom is what you are allowed to send

lossless gets you to half. you need a fiftieth.

the same parrot, four times



png · 93 107 bytes
lossless



jpg · 10 587 bytes
quality 75



jpg · 3 081 bytes
quality 10



jpg · 2 032 bytes
128 × 128 points, quality 25

still a parrot. and now it fits.

real files, sizes as measured · image: ai-generated (gpt-image-2)

the picture

two roads to 2 kb



256 × 256 · quality 1 · 1 925 bytes

all the points, each one coarse



128 × 128 · quality 25 · 2 032 bytes

a quarter of the points, each one clean

fewer points, or coarser points. analog and digital, from the other side.

a one-way street

lossless: unpack, and you hold the original

lossy: unpack, and you hold something that looks like it

from the 2 kb parrot, the 93 kb never come back.

lossless or lossy

must exactly this file arrive? then lossy is out.

must a presentable picture arrive? then it is your strongest lever.

the task decides, not the tool. and ask before you spend an hour optimising.

the receiver has to know

which method

which settings

written into the specification, next to preamble and character code

packed data does not explain itself.

today

- 1 zip three files of your own and explain the three ratios
- 2 bring the parrot under 2 kb, and write down what you gave up
- 3 put method and settings into your specification

two kilobytes.
still a parrot.



photo: parrot, 128 × 128 points, 2 032 bytes, shown point for point · image: ai-generated (gpt-image-2)

the picture